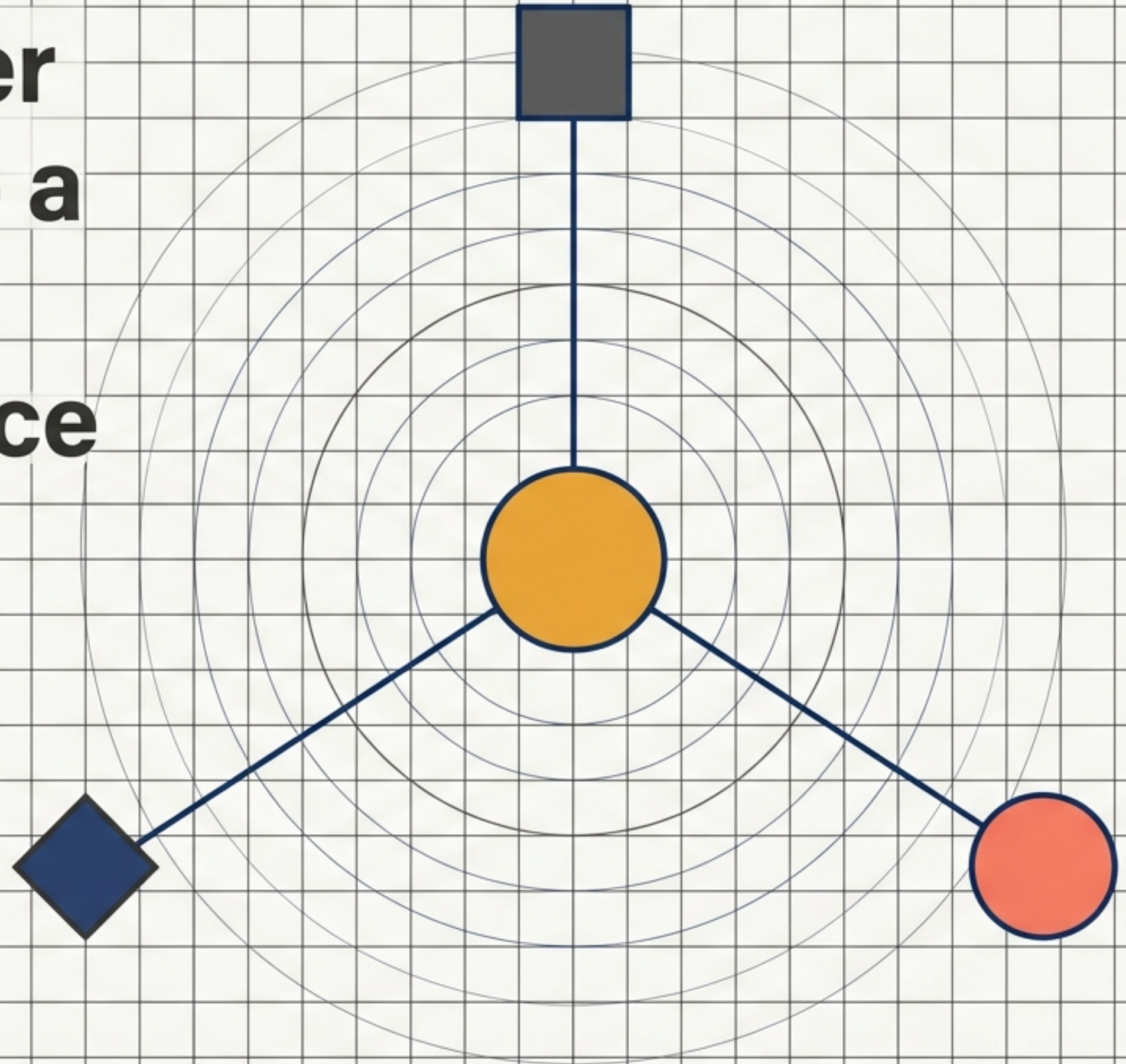


Turning volunteer dog walking into a manageable, repeatable service

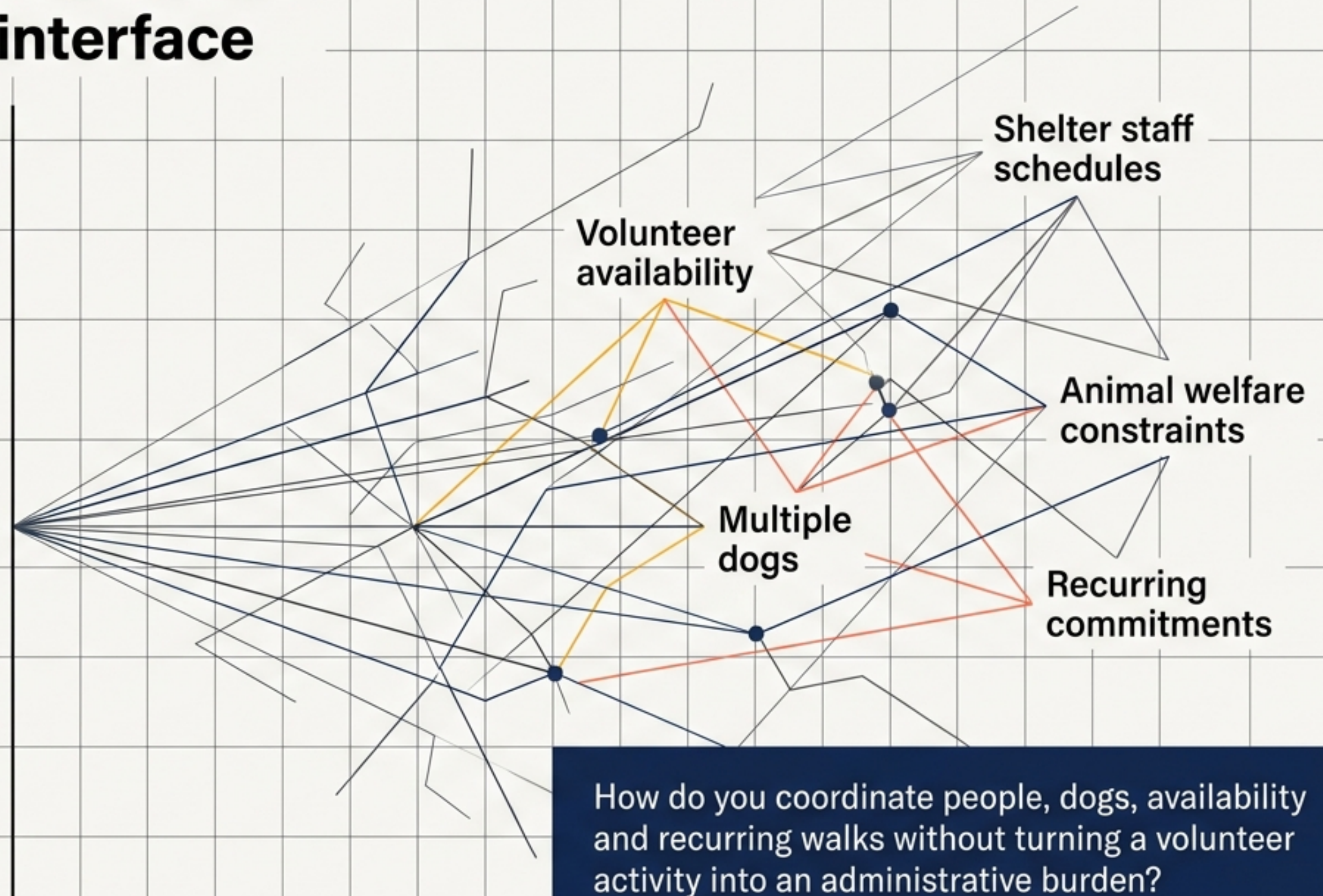
A product design case study for DARG (Domestic Animal Rescue Group), Hout Bay, Cape Town.



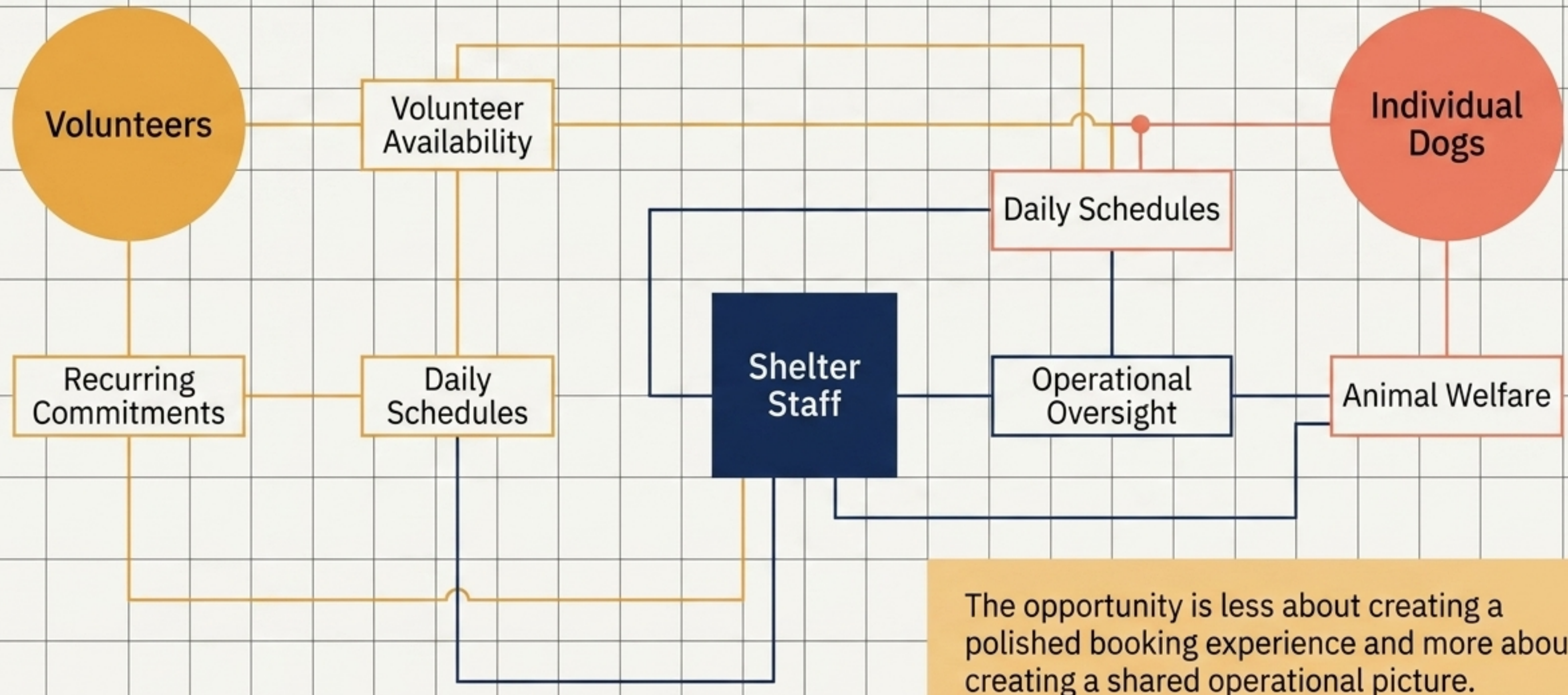
The challenge appears to be a simple booking interface

DARG requires volunteers to complete an orientation before walking dogs. The initial assumption is a simple app where a user books a dog.

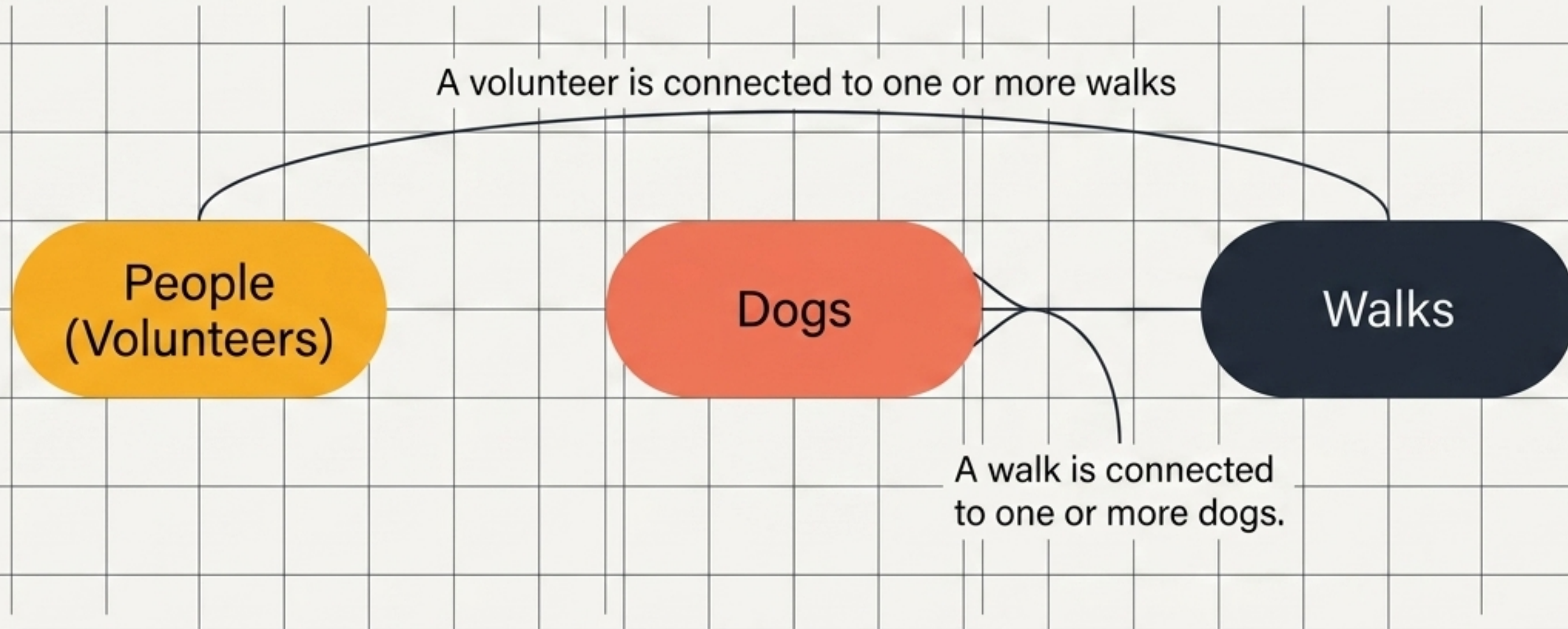
Book a Dog



Rescue environments possess an entirely different set of operational constraints



The product acts as a lightweight operational layer



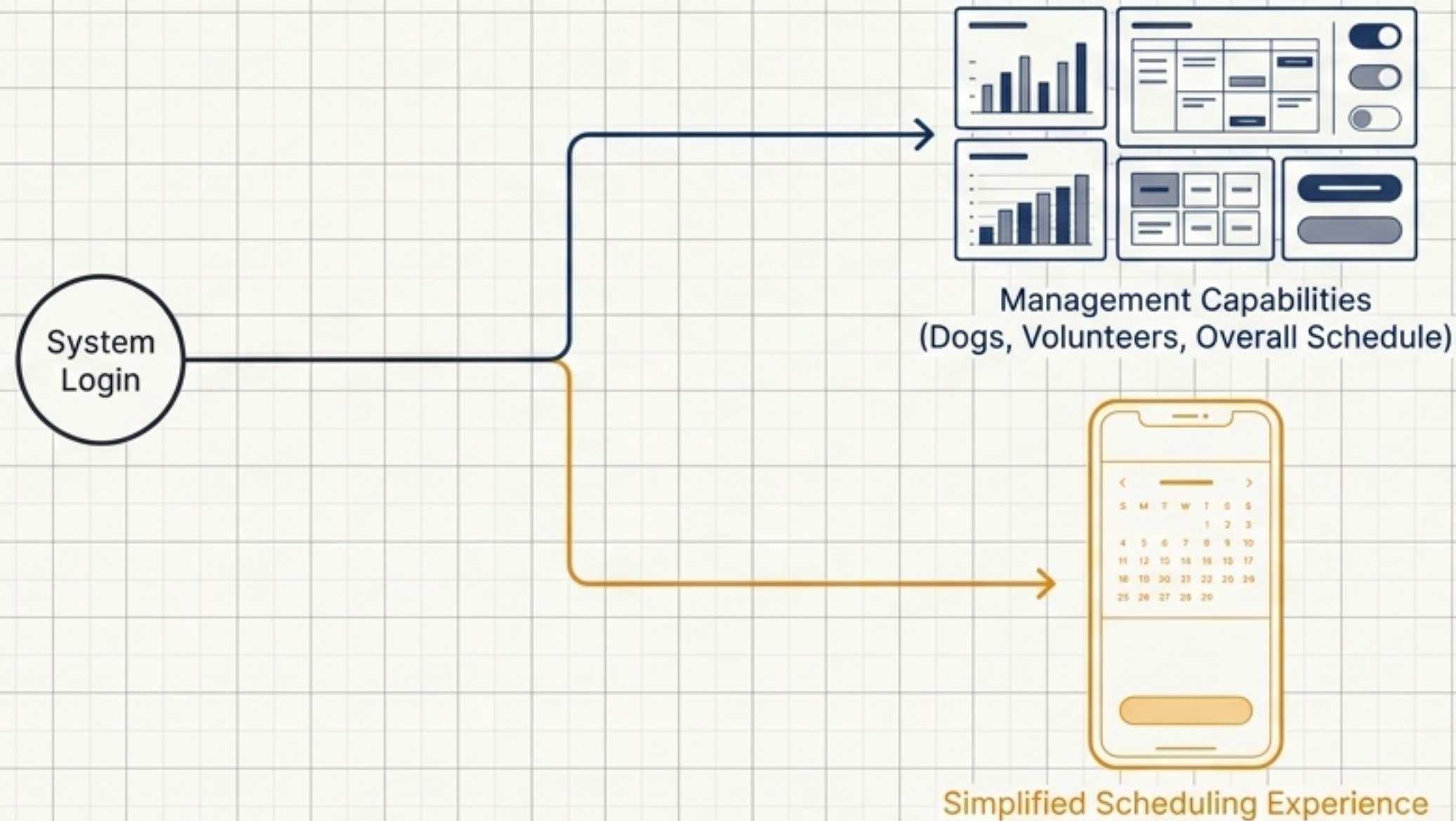
These three core entities form the backbone of the experience.
The product sits between the volunteers and the organisation.

Administrators and volunteers think about the exact same system differently

The Volunteer View		The Administrator View	
Focus:	Micro/Personal	Focus:	Macro/Organizational
Question:	When can I walk a dog?	Question:	What is happening across the organisation?
Mindset:	My commitments, my dogs, my schedule, available opportunities, avoiding unnecessary administration.	Mindset:	Operational overview, conflict resolution, managing volunteers/dogs, maintaining operational control.

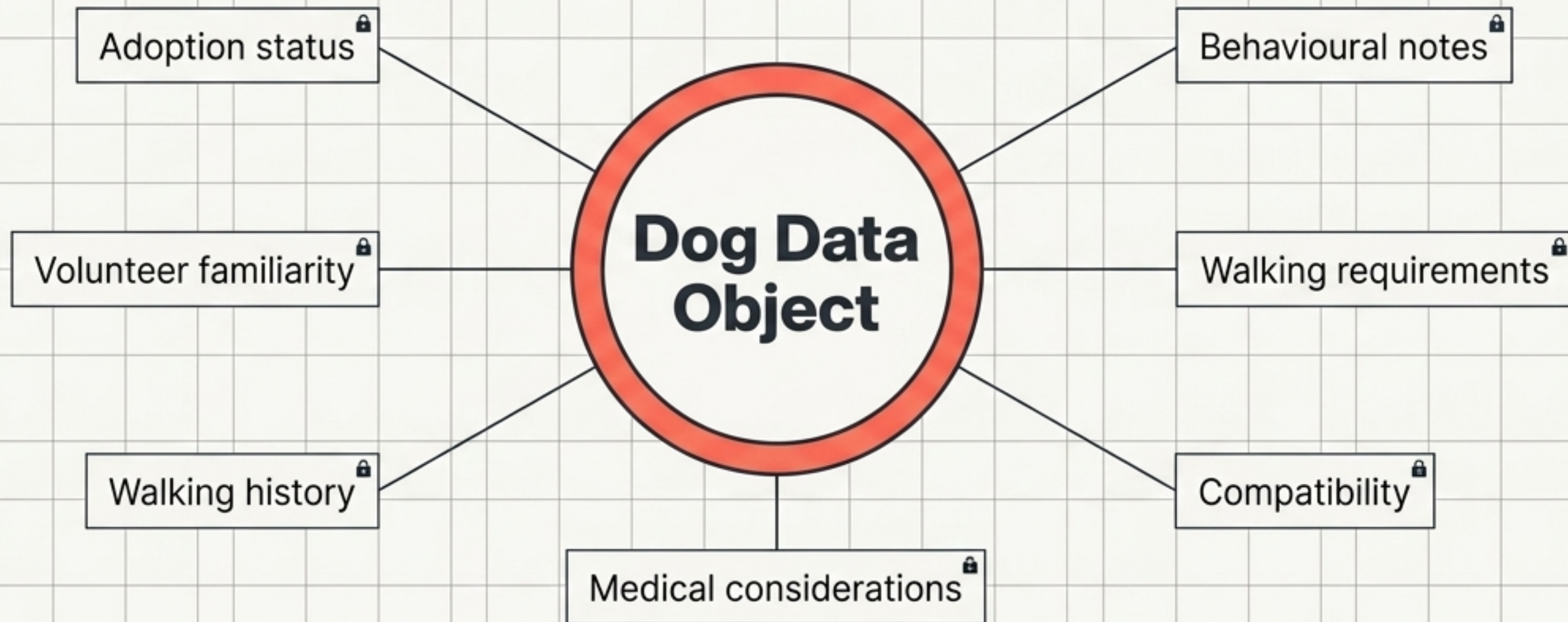
The system must support both mental models simultaneously without cross-contamination.

Give people the control they need, not the complexity the system contains



This creates two different levels of responsibility without forcing either user group to navigate functionality they don't need.

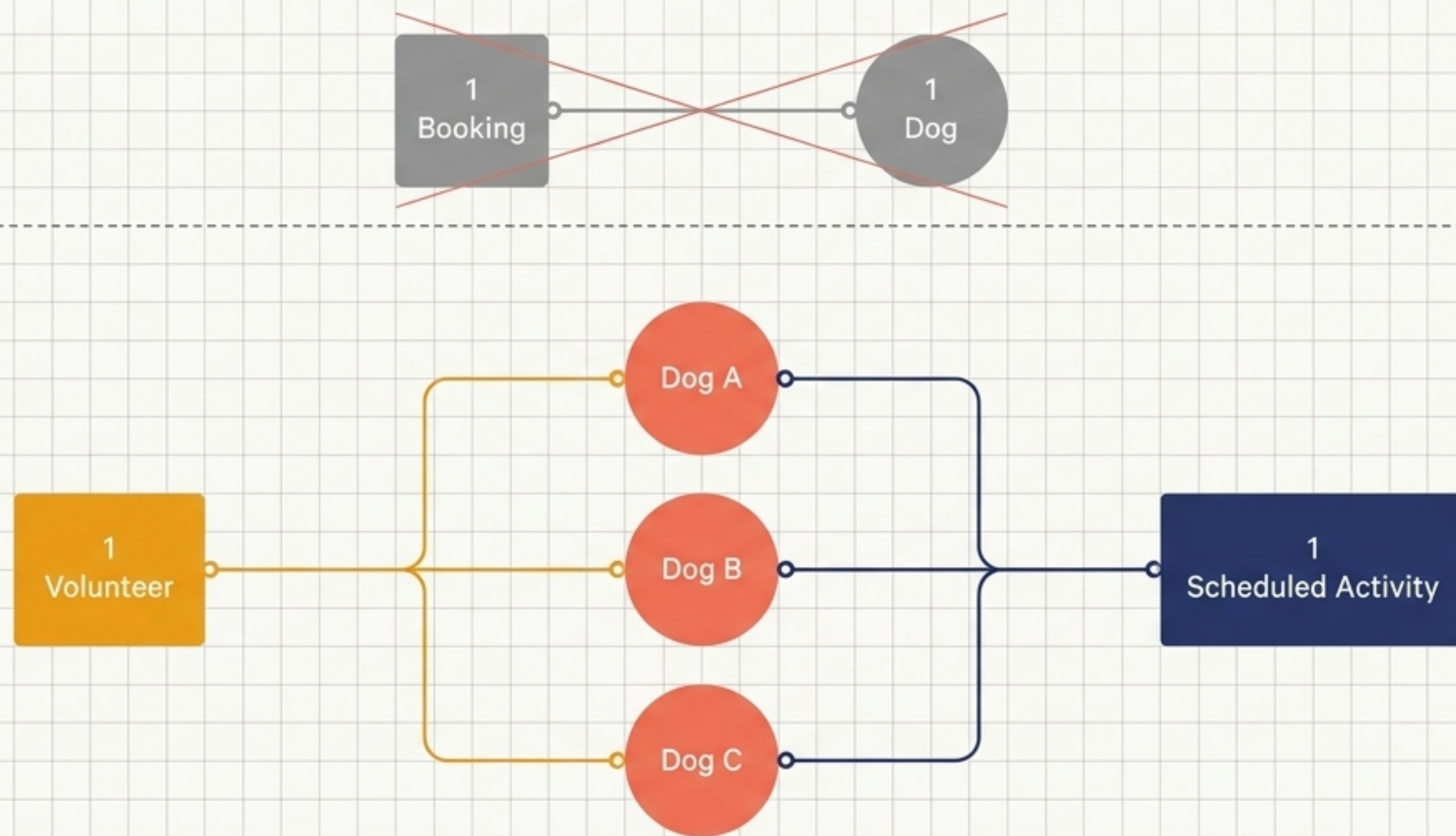
Dogs are treated as first-class objects, not calendar labels



Establishing this structure early prevents a flat booking database that would become difficult to extend.

It creates a central source of truth for the animals being scheduled and managed.

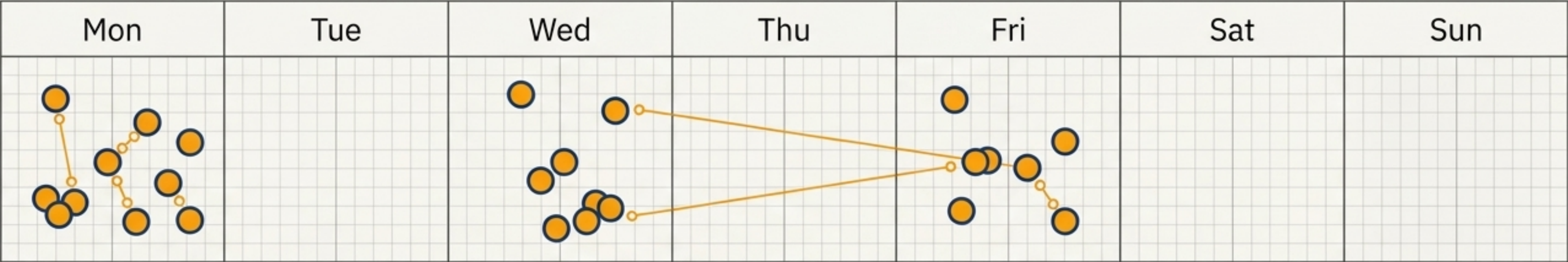
Rejecting the assumptions of commercial platforms



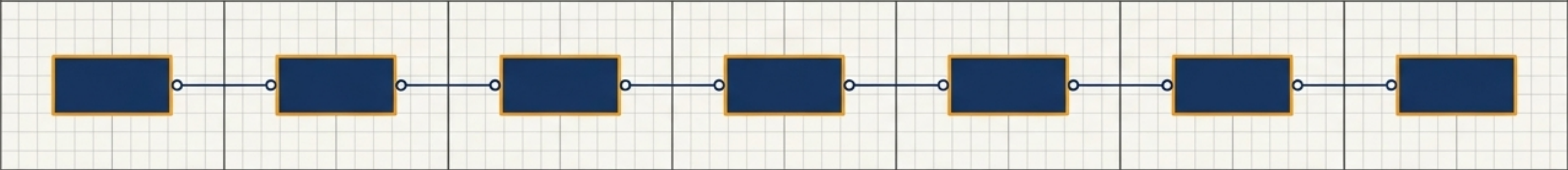
Volunteers may want to walk more than one dog at a time. The system creates a realistic model of shelter operations: one volunteer → multiple dogs → one scheduled activity.

Transforming ad-hoc volunteering into predictable capacity

“I’ll see when I’m free next week.”



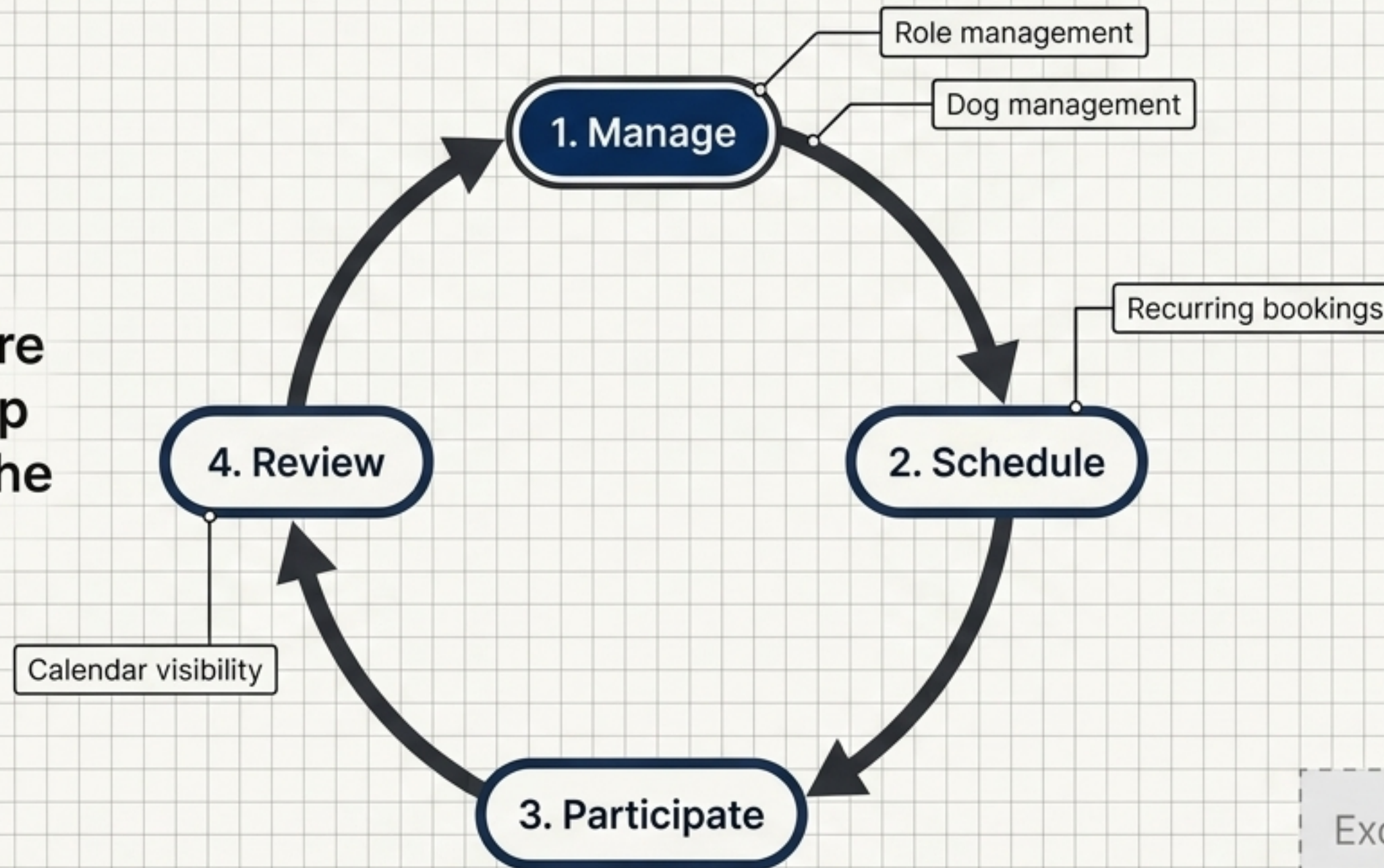
“I walk this dog every Tuesday at 10:00.”



Recurring scheduling is considered a first-class interaction. The interface must support this behaviour rather than forcing repetitive data entry.

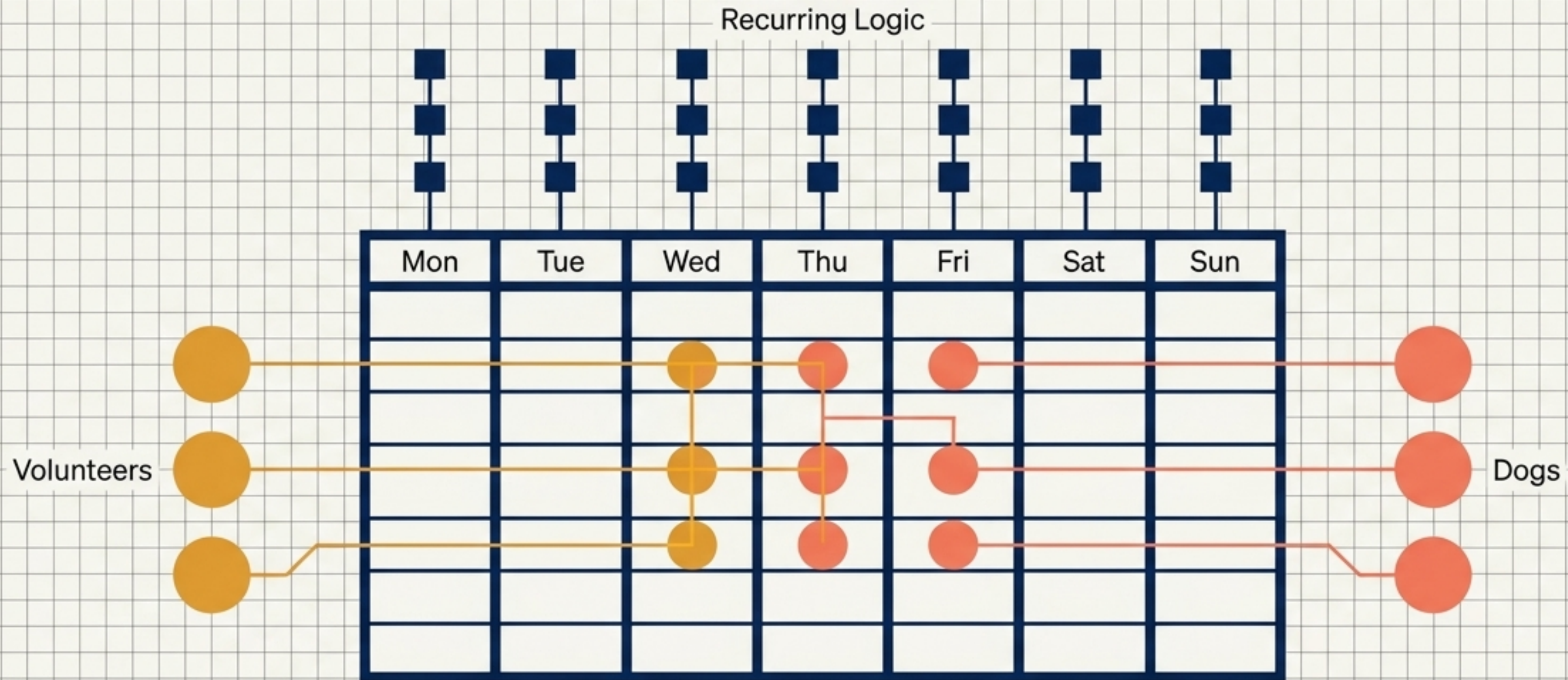
Defining the MVP around the smallest useful operational loop

Excluded features are kept outside the loop to prevent diluting the core experience.



Excluded from MVP:
messaging, payments,
advanced reporting.

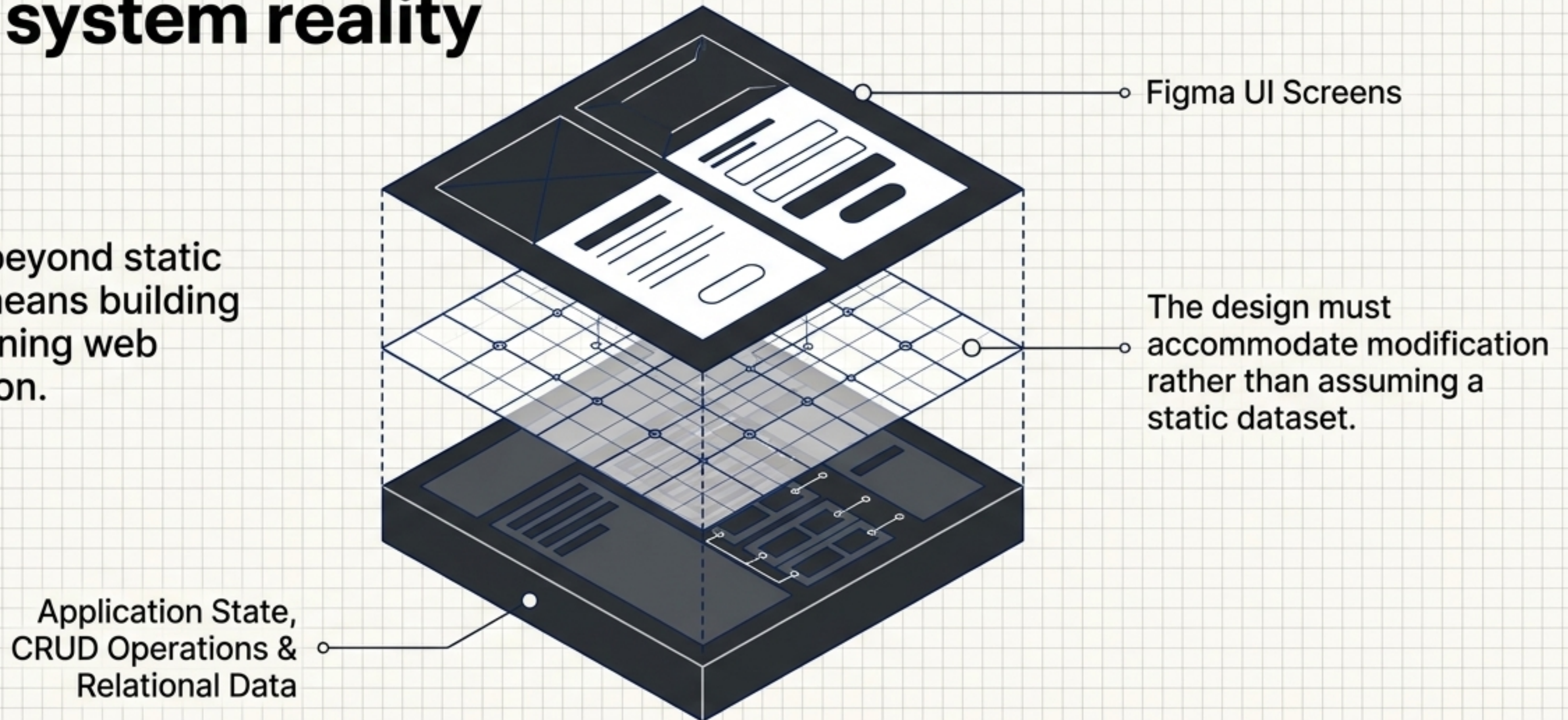
The calendar acts as the shared operational surface



Core Insight: The calendar is not a date picker. It is the organisation's operational timeline. It visualises capacity, responsibility, and conflicts, acting closer to a lightweight dispatch system than a traditional booking app.

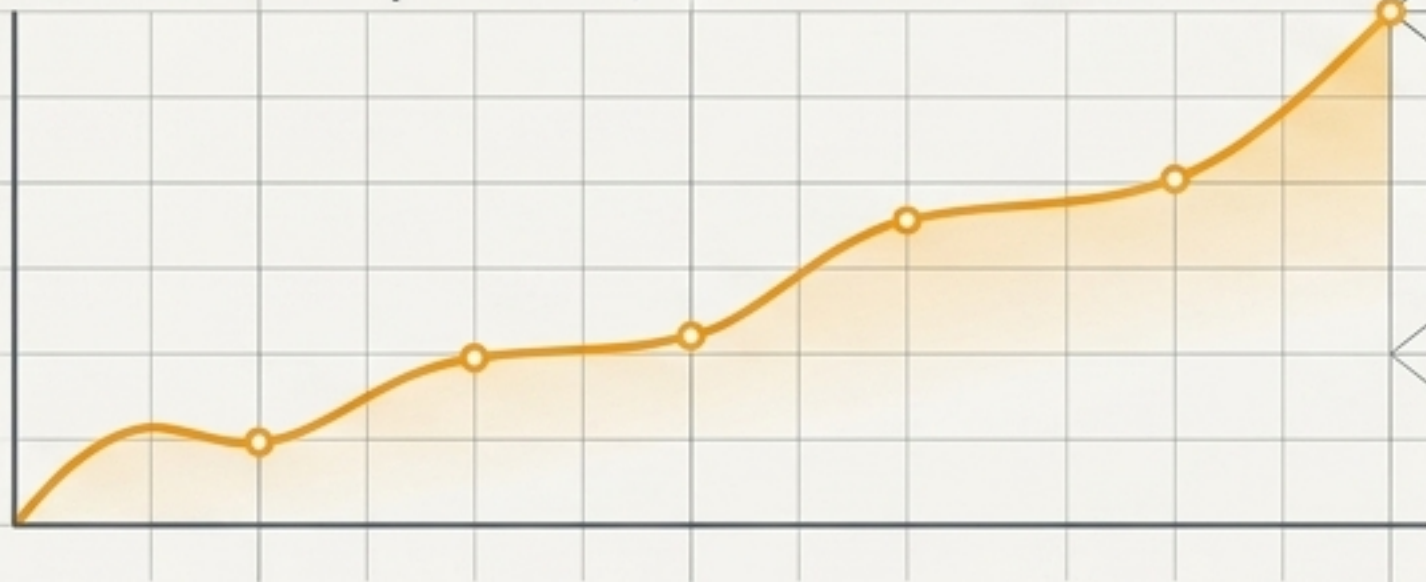
Interaction decisions must survive contact with system reality

Moving beyond static design means building a functioning web application.

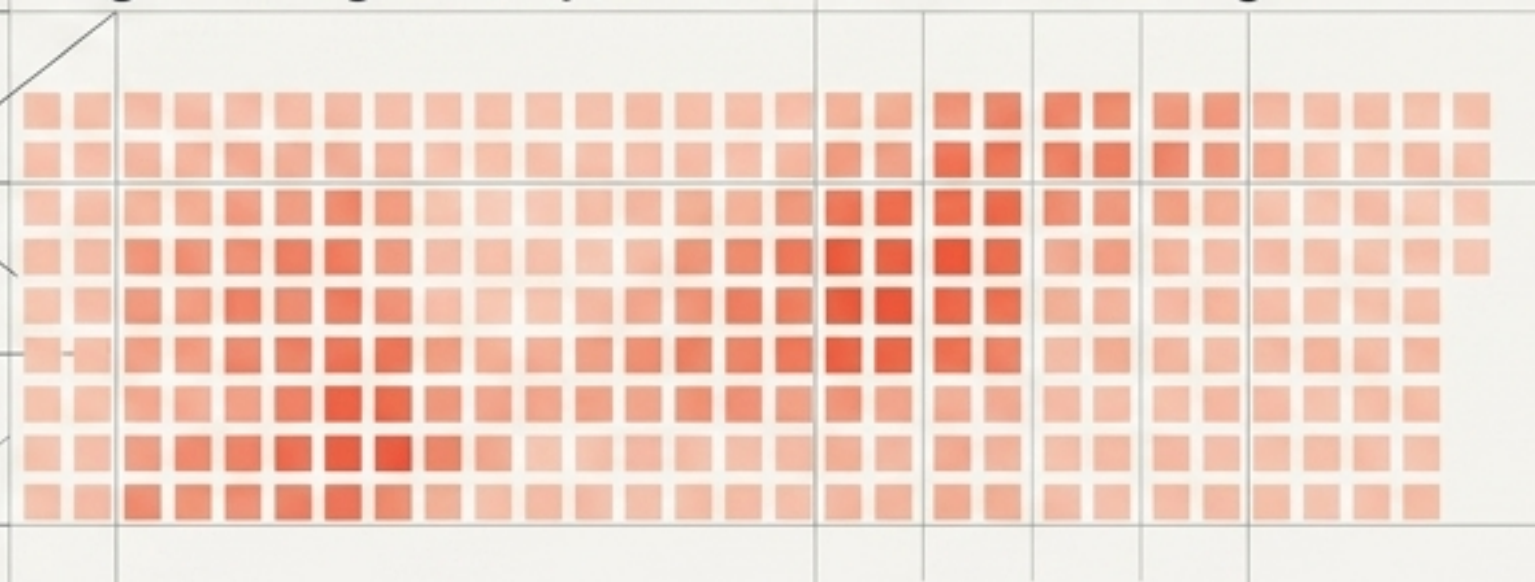


The long-term opportunity is operational intelligence

Volunteer Patterns: Booking friction, cancellation patterns, and retention.

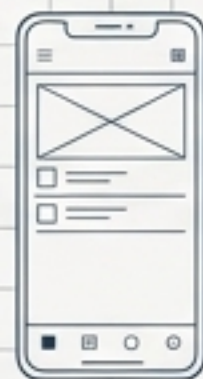


Animal Welfare: Behavioural constraints, individual dog walking history, and under-served dogs.

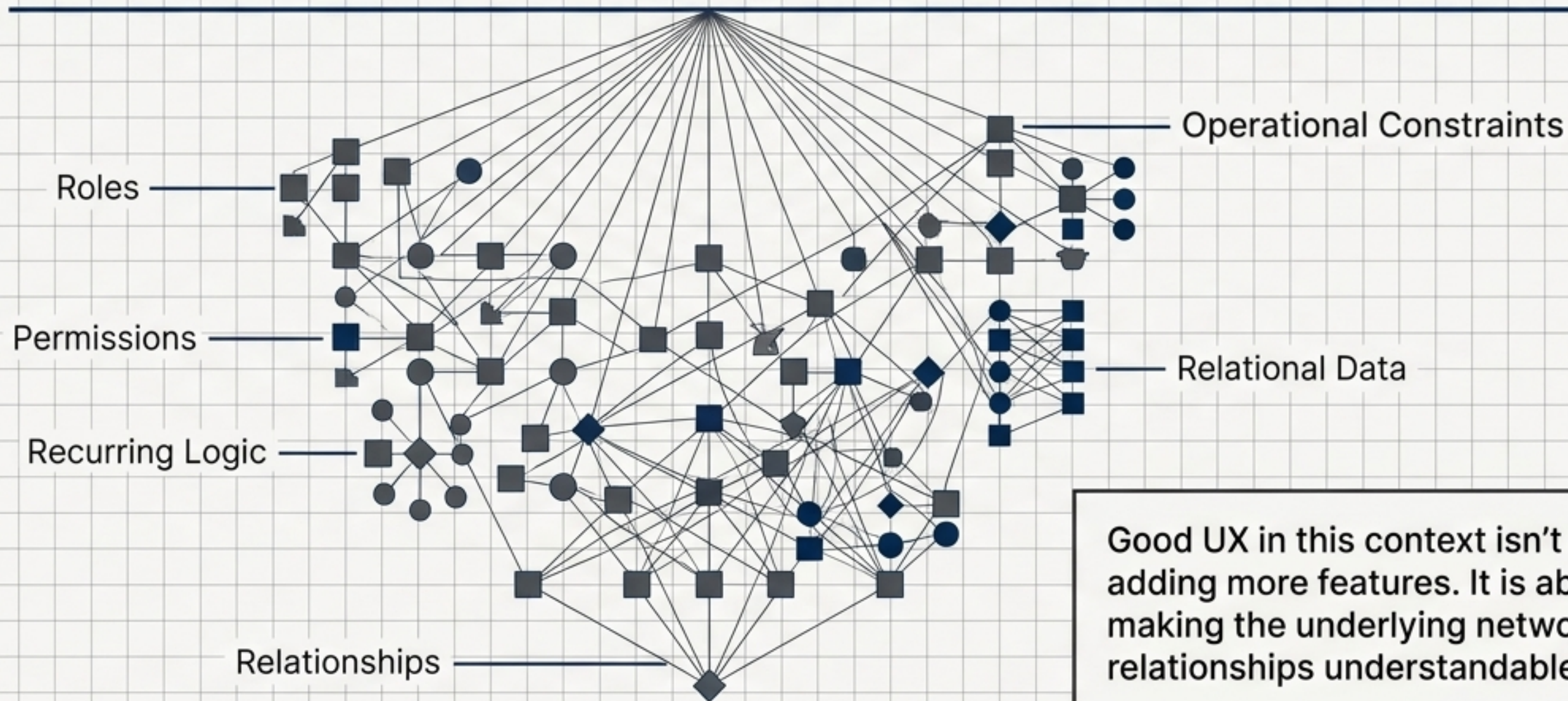


The goal is not simply better booking, but better visibility into the relationship between volunteer capacity and animal needs.

The interface is often only the visible tip of the system



The Waterline



Good UX in this context isn't about adding more features. It is about making the underlying network of relationships understandable.